

QATIP Intermediate

GCP Lab 03

Setting Up Secret Manager and IAM Roles for Terraform Authentication

Lab Objectives

This lab demonstrates secure methods for authenticating Terraform on GCP, avoiding direct use of long-lived credentials. You will:

- Store GCP service account credentials securely in Secret Manager
- Use those credentials to authenticate Terraform
- Alternatively, use Workload Identity Federation for secure, short-lived authentication

Teaching Points

Step 1: Administrator Tasks

- Set up GCP Secret Manager to store Terraform service account credentials
- Create a minimal-permission service account for Terraform usage

Step 2: Developer Tasks

- Retrieve credentials from Secret Manager and export them securely
- Use Terraform with the retrieved credentials
- Optionally, configure and use Workload Identity Federation

Before You Begin

1. Complete Lab0 (GCP environment setup).
2. Set your working directory:

```
cd ~/gcp-tf-int/labs/03
```

3. Ensure required tools are installed:

```
sudo apt-get update && sudo apt-get install -y jq google-cloud-sdk
```

Administrator Sets Up GCP Secret Manager and Service Account

Create and Configure the Service Account

1. Create a service account:

```
gcloud iam service-accounts create terraform-user \
  --description="Service Account for Terraform" \
  --display-name="Terraform User"
```

2. Grant minimal permissions (Editor for lab purposes):

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:terraform-user@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/editor"
```

3. Create a key for the service account:

```
gcloud iam service-accounts keys create terraform-user-key.json \
  --iam-account=terraform-user@$PROJECT_ID.iam.gserviceaccount.com
```

4. Store the key in Secret Manager:

```
gcloud secrets create terraform-gcp-key \
  --data-file=terraform-user-key.json
```

Developers Retrieve GCP Credentials from Secret Manager

Retrieve the Credentials

1. Fetch the secret from Secret Manager:

```
gcloud secrets versions access latest \
  --secret="terraform-gcp-key" > terraform-user-key.json
```

2. Set the environment variable for Terraform:

```
export GOOGLE_APPLICATION_CREDENTIALS=~/.gcp-tf-int/labs/03/terraform-user-key.json
```

3. Verify authentication:

```
gcloud auth activate-service-account --key-file=$GOOGLE_APPLICATION_CREDENTIALS
gcloud auth list
```

Run Terraform with Retrieved Credentials

1. Edit main.tf and set a unique bucket name or resource ID.
2. Ensure credentials are active:

```
source ./set-creds.sh # if scripted
```

3. Initialize Terraform:

```
terraform init
```

4. Plan the deployment:

```
terraform plan
```

5. Apply the deployment:

```
terraform apply
```

6. Verify resources in the GCP Console.

Optional: Using Workload Identity Federation

This method avoids storing service account keys.

1. Set up Workload Identity Pool and Provider.
2. Allow your external identity (e.g., GitHub Actions, OIDC) to impersonate the Terraform service account.
3. Use `gcloud auth workload-identity-federation` commands for authentication in CI/CD.

Lab Clean-up

1. Destroy Terraform resources:

```
terraform destroy
```

2. Optionally delete:

```
gcloud secrets delete terraform-gcp-key
gcloud iam service-accounts delete terraform-
user@$PROJECT_ID.iam.gserviceaccount.com
rm terraform-user-key.json
```

 Congratulations! You've completed Lab 03 for Terraform Authentication on Google Cloud.